

高级语言程序设计实验报告

南开大学计算机大类

姓名： 尹灿宇

学号： 2513617

班级： 3137

目录

一、 作业题目	2
二、 开发软件	2
三、 课题要求	2
3.1 核心任务	2
3.2 技术指标	2
3.3 AI 工具使用披露	2
四、 主要流程	3
4.1 具体开发步骤	3
4.2 代码架构	4
4.3 算法与数学公式	4
五、 单元测试	6
5.1 测试案例定义	6
5.2 测试结果与 Bug 修复	6
六、 收获	6
七、 总结	6

一、 作业题目

本项目开发了一款基于 C++ 的双人同屏动作对战游戏，命名为《空洞骑士 VS 大黄蜂》。玩家可以在同一台设备上分别控制小骑士（Knight）和大黄蜂（Hornet），利用各自独特的技能机制进行即时战斗对抗。

二、 开发软件

1. 开发环境：Visual Studio 2022
2. 图形化平台：SFML (Simple and Fast Multimedia Library)
3. 辅助工具：Gemini (辅助排查环境配置及物理逻辑优化)

三、 课题要求

3.1 核心任务

根据课程要求，自主选择题目并使用 C++ 语言完成一个具有图形化界面的程序。本项目旨在通过动作游戏的开发，实践面向对象编程（OOP）、物理碰撞检测及实时图形渲染等核心技术。

3.2 技术指标

1. 面向对象设计：通过定义虚基类提取角色共性，利用继承与多态实现不同角色的差异化逻辑。
2. 物理与逻辑检测：手写重力模拟公式、AABB 矩形碰撞检测算法，并使用状态机管理角色动作切换。
3. 多媒体应用：基于 SFML 实现逐帧动画渲染，并处理多通道背景音乐与音效的混合。

3.3 AI 工具使用披露

开发过程中使用 Gemini 协助解决了以下技术难点：

1. 链接器报错排查：解决了配置 sfml-audio 模块时出现的 LNK2019 错误，明确了在“附加依赖项”中添加库文件的必要性。
2. 物理逻辑优化：梳理了大黄蜂技能释放时的滞空重力累加逻辑，并协助校准了视觉特效与物理碰撞框的适配参数。

四、 主要流程

为了更清晰地展示代码是如何一步步跑起来的，我将这部分拆分为具体开发步骤、代码架构以及核心算法三个层面进行详述。

4.1 具体开发步骤

整个项目的开发其实就是一个从零开始、不断踩坑并调优的过程，主要经历了以下五个阶段：

(1) 环境搭建第一步是在 Visual Studio 2022 中配置 SFML 图形库。最开始只是想弄出一个能保持 60 帧运行的窗口，并把《空洞骑士》的背景图 (background.png) 铺上去。在这个阶段，我确立了游戏最核心的 while (window.isOpen()) 死循环架构，也就是所有动作游戏必备的“事件捕获——逻辑更新——画面清空与重绘”的基本框架。

(2) 角色基类与“防穿模”系统有了画面后，下一步是让人物站得住。为了避免代码重复，我写了一个 BaseCharacter 基类，把角色的坐标、血量 (hp)、无敌时间 (wudi_time) 以及向下的重力速度 (v_y) 全放了进去。最开始测试的时候小骑士总是掉出屏幕，于是我加了地板碰撞检测：一旦角色的 Y 轴坐标加上自身高度超过了预设的 FLOOR_Y，就把速度强制清零并把角色拉回地面。这一步确保了角色有扎实的落地感。

(3) 状态机与动画的防抽搐为了让小骑士和大黄蜂能跑能跳，我给他们分别建了子类，并往里面塞了大量的贴图数组 (std::vector<sf::Texture>)。一开始我直接把切图逻辑写在了帧循环里，结果由于电脑 CPU 跑得太快，角色跑动起来像在疯狂抽搐。为了解决这个问题，我引入了一个简易的 status 状态机（用 0、1、3 分别代表待机、跑动、攻击），并且加了一个名为 t_anim 的时间累加器。让它每帧缓慢增加，只有当累加值超过设定的阈值（比如 0.08 秒）时，画面才切到下一帧，这才把角色的动作彻底平滑下来。

(4) 战斗判定与视觉动作游戏最关键的就是打击感。我没有用现成的物理引擎，而是直接手写判定。小骑士的平砍，我就在代码里临时生成一个 sf::FloatRect 矩形，看看它有没有和敌人的身体重叠；大黄蜂的丝线大招，我就直接算两点之间的距离。在这期间我遇到了一个大坑：大黄蜂大招的白色特效被我放大了两倍，但底层判定的有效距离没改，导致“看着打中了但其实没掉血”。后来我手动把距离判定的阈值拉大到了 350 像素点，做了一波“视觉补偿”，手感瞬间就扎实了。小骑士的下劈弹跳逻辑也是在这个阶段抠细节抠出来的。

(5) 技能 CD、音效与结算界面开发到后期，室友在试玩时发现大黄蜂可以无 CD 无限放技能，能在天上一直飞，小骑士毫无还手之力。于是我立刻加了 dash_cd 和 silk_cd 的冷却计时器，并给大黄蜂施法时加了硬直锁。最后一步是给游戏注入灵魂的音频混合，正是在给大黄蜂加“Adido”、“Heigali”配音时，我遭遇了满屏的 LNK2019 报错，最终发现是漏加了 sfml-audio.lib。音效调通后，我又顺手加上了血条 UI、镜头动态跟

随和判定生死的结算画面，游戏至此终于形成了一个完整的闭环。

4.2 代码架构

本项目的代码没有把所有逻辑都堆在 `main.cpp` 里，而是采用了一种“模块化”的思路，将整个系统分成了底层的基类模板层、中层的角色实现层以及顶层的逻辑驱动层。

(1) 基类模板层 (`BaseCharacter`) 在设计初期，我意识到小骑士和大黄蜂虽然技能完全不同，但他们都要跳跃、都要受重力影响、都有血量。于是我定义了 `BaseCharacter` 类作为所有角色的“祖先”。

属性封装：这里存放了最基础的物理数据，比如 `v_y`（垂直速度）和 `on_ground`（落地状态）。这样写的好处是，以后如果我想再加一个角色（比如守望者），直接继承这个类就行，不用重新写一遍重力代码。

虚函数的力量：我把 `updatePhysics` 定义成了虚函数。这对我来说非常关键，因为虽然大家都要更新物理状态，但大黄蜂有“滞空”和“冲刺”这种特殊物理，通过虚函数，我可以在主循环里用同一个基类指针指向不同的角色，程序会自动根据角色身份去跑对应的逻辑，这就是课上讲的多态。

(2) 角色实现层 (`Knight & Hornet`) 这两个派生类是游戏的灵魂。

状态机驱动：我给每个子类都设计了一个 `int status` 变量作为状态机。比如在 `Knight` 类里，`status` 为 3 时代表正在“平砍”，为 4 时代表“下劈”。

资源解耦：每个子类各自管理自己的图片数组 (`std::vector<sf::Texture>`)。比如 `Hornet` 类里有专门存放丝线特效的 `silk_eff_tex`。这种设计避免了资源混乱，每个角色只需要管好自己的动画序列，互不干扰。

(3) 逻辑驱动层 `main.cpp` 扮演的是“导演”的角色。

三段式循环：它严格按照“输入响应—逻辑更新—画面渲染”的顺序执行。我特意把物理更新 (`updatePhysics`) 放在了碰撞检测之前，这样能保证判定框始终跟着角色的最新位置走，不会出现“人已经过去了，判定还在后面”的延迟感。

相机与 UI 的分离：我使用了 `sf::View` 来实现动态镜头，但 UI 血条的绘制却是在默认视角下进行的。这种“双视角”架构保证了无论两个角色怎么打、镜头怎么缩放，顶部的血条始终固定在屏幕上方，这也是从现代商业游戏里学到的架构经验。

4.3 算法与数学公式

其实在这个项目里，我并没有用到什么高深的第三方物理引擎。游戏里所有的“打击感”和“重量感”，全是我利用基础的运动学公式和解析几何一行行手敲出来的。

(1) 重力模拟与防穿模

动作游戏如果没有重力，角色就会像飘在空中的纸片人。为了模拟真实的下落感，我在 `BaseCharacter::updatePhysics` 中给角色施加了每帧向下的加速度。

角色的垂直速度 v_y 和纵坐标 y 遵循以下迭代公式：

$$v_y = v_y + G \quad (\text{代码中我设定的重力常数 } G = 1.2f)$$

$$y_{new} = y_{old} + v_y$$

为了防止角色一直往下掉“穿模”掉出屏幕，我紧接着做了一个地板检测：当检测到角色底部的坐标超过了设定的地板线 ($y + height \geq floorY$) 时，强行把角色的 Y 坐标拉回地面，并将 v_y 清零，这就形成了极其扎实的落地判定。

(2) AABB 矩形碰撞判定

小骑士的平砍和下劈，本质上是在角色正前方（或脚下）瞬间生成一个肉眼看不见的矩形判定框（Hitbox）。这里使用的是 AABB（Axis-Aligned Bounding Box，轴对齐包围盒）算法。

判断两个矩形是否重叠的底层数学逻辑是：

$$\text{Hit} = (R_{left} < B_{right}) \wedge (R_{right} > B_{left}) \wedge (R_{top} < B_{bottom}) \wedge (R_{bottom} > B_{top})$$

为了简化代码，我直接调用了 SFML 提供的 intersects() 函数。当小骑士按下攻击键且正好处于特定动画帧时，只要这个生成的临时攻击框与大黄蜂的身体矩形相交，且大黄蜂不处于无敌帧 ($wudi_time \leq 0$)，就会成功扣血并产生击退位移。

(3) 欧几里得距离判定

大黄蜂的“丝线风暴”是一个圆形的范围伤害。如果用上面的矩形判定，会导致四个角有误差（明明在角落没碰到特效却掉血了）。因此，我改用了两点间的直线距离公式来判断伤害范围。

假设大黄蜂的中心坐标为 (x_1, y_1) ，小骑士的中心坐标为 (x_2, y_2) ，计算距离：

$$Distance = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

在实际开发中，我发现如果判定半径设得太小，就会出现“白色的丝线特效已经刮到小骑士脸上了，但他却不掉血”的诡异现象。于是我在这里做了一个视觉补偿：强行把伤害判定的阈值拉大到了 350 个像素点 ($\text{if } (Distance < 350.0f)$)，确保底层的物理判定框和放大了两倍的视觉特效完美重合，手感一下子就扎实了。

(4) 镜头算法

为了让游戏视角始终跟着两个角色的中心点走，同时又不能让镜头飞出背景地图的黑边，我写了一套简单的边界限制（Clamp）逻辑。

以镜头的中心点 X 坐标 (tx) 为例，通过判断并强制赋值，确保它不低于地图左边缘警戒值，也不高于右边缘警戒值：

```
if (tx < vw / 2.0f) tx = vw / 2.0f;
```

```
if (tx > MAP_WIDTH - vw / 2.0f) tx = MAP_WIDTH - vw / 2.0f;
```

这几行简单的 if 判断构成了摄像机的“空气墙”，彻底规避了游戏运行中背景露黑边的尴尬现象。

五、 单元测试

5.1 测试案例定义

表 1: 游戏核心逻辑测试案例

玩家操作	预期程序现象	测试目的
小骑士靠近并平砍	大黄蜂血量扣除，产生后退位移并变色。	测试 AABB 碰撞判定。
空中对准目标下劈	目标受损，小骑士获得向上的瞬时速度。	测试下劈 Hitbox 逻辑。
大黄蜂释放丝线技能	范围内目标受到伤害，大黄蜂短暂滞空。	测试圆形 AOE 判定。
连续触发冲刺键	仅首次生效，控制台打印冷却中提示。	测试技能 CD 拦截机制。

5.2 测试结果与 Bug 修复

在测试过程中发现大黄蜂大招的视觉效果与实际判定不匹配。经代码审计发现，由于特效贴图进行了 2 倍缩放 (setScale(2.0, 2.0))，导致物理框覆盖不足。通过将判定半径从 260.0f 提升至 350.0f，成功实现了物理反馈与视觉表现的统一。

六、 收获

- 1. 内存管理：深刻理解了虚析构函数在多态设计中的“保命”作用，有效规避了因资源未清理导致的内存泄漏风险。
- 2. 编译机制：理清了头文件声明与.lib 静态库实现之间的工程逻辑，掌握了 LNK2019 报错的解决流程。
- 3. 数学工程化：通过将坐标几何与物理公式转化为实时的游戏反馈，增强了对数学模型向工程实体转译的理解。

七、 总结

本项目从零开始构建了一个功能完整的双人对战程序。在解决穿模、状态切换、环境配置等具体问题的过程中，我不仅巩固了 C++ 编程能力，更通过面向对象架构的设计，体会到了软件工程中结构化管理代码的优势。